## How to Execute Task 3: Zipf's Law with NLTK

This task demonstrates **Zipf's Law**, which states that the frequency of a word is approximately inversely proportional to its rank:

For example:

| Rank | Frequency |
|---|---|
| **1** | 1000 |
| **2** | 500 |
| **3** | 333 |
| **4** | 250 |

## Step 1: Install Required Libraries

Open Command Prompt or Terminal and run:

```
pip install nltk matplotlib
```

## Step 2: Download the Gutenberg Corpus

Create a Python file named zipf_law.py and run the following once:

```
import nltk

nltk.download('gutenberg')
nltk.download('punkt')
```

This downloads the corpus used in the experiment.

## Step 3: Copy the Program

```
import nltk
import collections
import matplotlib.pyplot as plt

nltk.download('gutenberg')

words = [
    w.lower()
    for w in nltk.corpus.gutenberg.words("austen-emma.txt")
 ]

```

```
freq = collections.Counter(words).most_common()

ranks = range(1, len(freq) + 1)

plt.loglog(
    ranks,
    [f for _, f in freq]
)

plt.xlabel("Rank")
plt.ylabel("Frequency")
plt.title("Zipf's Law for Emma")
plt.show()

print(freq[:10])
```

## Step 4: Run the Program

```
python zipf_law.py
```

## Expected Output

### 1. Graph Window

A log-log graph appears showing:

- X-axis → Word Rank
- Y-axis → Word Frequency

If Zipf's Law holds, the curve should look approximately like a straight descending line.

---

### 2. Top Frequent Words

Example:

```
[
 ('the', 5201),
 (',', 5140),
 ('.', 3124),
 ('to', 3024),
 ('and', 2987),
 ('of', 2512),
```

```
  ('a', 2407),
  ('i', 2201),
   ('it', 2155),
   ('was', 2096)
  ]
```

The exact numbers may vary slightly depending on preprocessing.

---

## Interpretation

### Why does the plot become nearly straight?

The graph uses:

```
plt.loglog(...)
```

which means:

is plotted against

Zipf's Law predicts:

Taking logarithms:

This is the equation of a straight line.

---

## Exercise Answers

### 1. Is the log-log plot close to a straight line?

✅ Yes.

Natural language text generally follows Zipf's Law, so the graph should resemble a straight descending line.

---

### 2. Which words appear at the top?

Typically:

```
the
and
of
to
a
i
```

```
it
was
in
 that
```

These are called **function words** because they occur very frequently in English.

---

## 3. Would programming language keywords follow the same law?

✅ To some extent, yes.

Consider Python code:

```
if
for
while
def
class
return
import
```

Some keywords occur much more frequently than others.

Example:

```
if
=
(
)
:
for
def
```

will appear more often than

```
yield
global
nonlocal
```

Therefore source code often exhibits a Zipf-like distribution, although the pattern is usually less perfect than natural language because programming languages have stricter syntax.

---

## Enhanced Version (Ignoring Punctuation)

To analyze only words:

```
words = [
    w.lower()
    for w in nltk.corpus.gutenberg.words("austen-emma.txt")
    if w.isalpha()
]
```

This removes punctuation such as:

```
.
,
;
:
!
?
```

and produces a cleaner Zipf curve.

---

## Key Learning Outcome

- **Frequency Distribution** measures how often words appear.
- **Zipf's Law** states that word frequency is inversely proportional to rank.
- The **log-log plot** transforms this relationship into an approximately straight line.
- Human languages, programming languages, web data, and many social systems often exhibit Zipf-like behavior.